

Week 3: Intermediate Dart Programming

Dr M AOUE

Lecture notes

Objectives. By the end of this lecture you will be able to write a Dart class with fields, methods and constructors; create objects; protect a field with a private name, a getter and a setter; print an object with `toString`; add a named constructor; and build a subclass with inheritance.

Every example of this lecture is a complete program in the folder `code/week03/slides`. The listings and outputs below are taken directly from those files, and each output was checked by running the program. See the guide *Running the programs in VS Code* to run them.

1 Object-oriented programming in Dart

Dart has many features that automate the creation of classes and objects. The ones we use in this lecture are:

- automatic initialization of class fields through the class constructor;
- implicit **getters and setters** for every class field;
- getters and setters are called automatically when you read or write a field;
- several **named constructors** for the same class.

2 A simple class

We create a class that represents a circle. First the Java version, which you already know:

```
1 public class Circle {
2     double radius;
3     // declare a constructor
4     public Circle(double r) {
5         radius = r;
6     }
7     // declare a method to return the area of the circle
8     public double getArea() {
9         return radius * radius * Math.PI;
10    }
11 }
```

The same class in Dart is much shorter:

- `Circle(this.radius);` is a constructor shorthand: it stores the argument in the field.
- Dart creates a getter and a setter for `radius` for you.
- An object is created by calling the class, `Circle(10)`: there is no `new` keyword.

```
1 import 'dart:math'; // gives us pi
2
3 class Circle {
4     double radius; // a field: each Circle object has its own radius
```

```

5
6 // Constructor. "this.radius" means: store the argument in the field radius.
7 Circle(this.radius);
8
9 // A method: returns the area of the circle.
10 double getArea() {
11     return pi * radius * radius;
12 }
13 }
14
15 void main() {
16     // Create a Circle object with radius 10 (no "new" needed).
17     // The 10 is written as an int but stored as a double: 10.0
18     Circle c1 = Circle(10);
19
20     // Reading the field calls its implicit getter.
21     print('Radius: ${c1.radius}');
22     print('Area: ${c1.getArea()}');
23
24     // Assigning to the field calls its implicit setter.
25     c1.radius = 20;
26     print('New radius: ${c1.radius}');
27     print('New area: ${c1.getArea()}');
28 }

```

Output:

```

Radius: 10.0
Area: 314.1592653589793
New radius: 20.0
New area: 1256.6370614359173

```

3 Private fields, getters and setters

A field whose name starts with an underscore, like `_radius`, is **private**. In Dart private means private to the *file* (the library), not to the class. That is why the `Circle` class is placed in its own file, `circle.dart`.

A private field is used through a **getter** and a **setter**. The setter can validate the new value: here a radius that is not positive is refused with an exception. The constructor also uses the setter, so an invalid circle can never be created. The same file also contains the `toString` method and the named constructor that are explained in the next section.

```

1 import 'dart:math'; // pi
2
3 class Circle {
4     // Private field: the underscore hides it from every other file.
5     // It starts at 0 so that it always has a value.
6     double _radius = 0;
7
8     // Main constructor. "radius = r" calls the SETTER below, so an invalid
9     // radius is rejected when the object is created.
10    Circle(double r) {
11        radius = r;
12    }
13
14    // Named constructor: an alternative way to create a Circle (radius 5).
15    // The part after ":" is the initializer list; it sets the field directly.
16    Circle.create() : _radius = 5;
17
18    // Getter: lets other files READ the private field as "c.radius".
19    double get radius => _radius;

```

```

20
21 // Setter: lets other files CHANGE the field as "c.radius = 20",
22 // but only if the value is valid.
23 set radius(double r) {
24     if (r > 0) {
25         _radius = r;
26     } else {
27         throw Exception('Invalid Radius');
28     }
29 }
30
31 double getArea() {
32     return pi * _radius * _radius;
33 }
34
35 // Called when the object is printed or used inside a string.
36 @override
37 String toString() {
38     return 'Radius: $_radius Area: ${getArea()}';
39 }
40 }

```

Another file imports the class and uses it through the getter and the setter:

```

1 import 'circle.dart'; // the library that defines Circle
2
3 void main() {
4     // The constructor calls the radius setter.
5     Circle c1 = Circle(10);
6     print('Radius: ${c1.radius}'); // calls the radius GETTER
7
8     // Calls the radius SETTER with a valid value.
9     c1.radius = 20;
10    print('After c1.radius = 20: ${c1.radius}');
11
12    // An invalid value: the setter throws an exception, which we catch.
13    try {
14        c1.radius = -2;
15    } on Exception catch (e) {
16        print('$e'); // Exception: Invalid Radius
17    }
18    print('Radius is still: ${c1.radius}'); // the bad value was refused
19
20    // The constructor is protected too.
21    try {
22        Circle(-1);
23    } on Exception catch (e) {
24        print('Constructor also validates: $e');
25    }
26
27    // Direct access to the private field does NOT compile in this file:
28    // c1._radius = 5; // Error: The setter '_radius' isn't defined for the type
29    // 'Circle'.
30 }

```

Output:

```

Radius: 10.0
After c1.radius = 20: 20.0
Exception: Invalid Radius
Radius is still: 20.0
Constructor also validates: Exception: Invalid Radius

```

Trying to use `c1._radius` from another file does not compile. The error says that the setter `_radius` is not defined for the type `Circle`.

4 The `toString` method and named constructors

`toString`

`toString` returns a text that describes the object. It is called automatically when you print the object or use it inside a string (`'$c1'`). If a class does not define it, the class inherits it from `Object`: all classes in Dart inherit from `Object`. Use `@override` to write your own version.

Named constructors

Dart does not have overloading, so a class cannot have two constructors with the same name. To offer alternative ways to build an object, Dart uses **named constructors**. In `circle.dart`, `Circle.create()` builds a circle of radius 5.

```
1 import 'circle.dart';
2
3 void main() {
4   // The main constructor: the radius is given by the caller.
5   Circle c1 = Circle(10);
6   print('c1 -> $c1'); // $c1 calls c1.toString()
7
8   // The named constructor: no argument, the radius is always 5.
9   Circle c2 = Circle.create();
10  print('c2 -> $c2');
11
12  // Calling toString() explicitly gives the same text.
13  print('Same text with toString(): ${c2.toString()}');
14 }
```

Output:

```
c1 -> Radius: 10.0 Area: 314.1592653589793
c2 -> Radius: 5.0 Area: 78.53981633974483
Same text with toString(): Radius: 5.0 Area: 78.53981633974483
```

5 Class inheritance

In addition to the implicit inheritance from `Object`, a class can extend another class with the `extends` keyword. It receives all the members of the parent class, except the constructors. The subclass constructor calls the parent constructor with `super(...)`, and a method can be overridden and still reuse the parent's version with `super.toString()`.

In the example `Shape` is the parent class and `Circle` the subclass. A `Circle` *is* a `Shape`, so it can be stored where a `Shape` is expected, and each object uses its own `toString`.

```
1 // The parent class (super class).
2 class Shape {
3   String color;
4
5   Shape(this.color); // sets the color field
6
7   @override
8   String toString() {
9     return 'Color: $color';
10  }
11 }
```

```

12
13 // The child class (subclass): everything in Shape, plus a radius.
14 class Circle extends Shape {
15     double radius;
16
17     // Circle(String color, this.radius) receives both values. After ":" the
18     // parent constructor is called to store the color.
19     Circle(String color, this.radius) : super(color);
20
21     @override
22     String toString() {
23         // super.toString() is the parent's text ("Color: red"); we add to it.
24         return '${super.toString()} Radius: $radius';
25     }
26 }
27
28 // The parameter is a Shape, but the object may really be a Circle.
29 // "is Circle" checks that; inside the if, Dart lets us use x.radius.
30 void whatIs(Shape x) {
31     if (x is Circle) {
32         print('This shape is a Circle with radius ${x.radius}');
33     } else {
34         print('This shape is a plain Shape');
35     }
36 }
37
38 void main() {
39     // A plain Shape.
40     Shape s = Shape('green');
41     print('Shape: $s');
42
43     // A Circle: the color is stored by Shape, the radius by Circle.
44     Circle c = Circle('red', 10);
45     print('Circle: $c');
46
47     // A Circle can be passed wherever a Shape is expected.
48     whatIs(s);
49     whatIs(c);
50
51     // A list of Shapes can hold both. For each object Dart calls ITS OWN
52     // toString(): this is called polymorphism.
53     List<Shape> shapes = [s, c];
54     print('All shapes in one list:');
55     for (final shape in shapes) {
56         print(shape);
57     }
58 }

```

Output:

```

Shape: Color: green
Circle: Color: red Radius: 10.0
This shape is a plain Shape
This shape is a Circle with radius 10.0
All shapes in one list:
Color: green
Color: red Radius: 10.0

```

6 Java and Dart at a glance

	Java	Dart
Create an object	<code>new Circle(10)</code>	<code>Circle(10)</code>
Store a constructor argument	<code>radius = r;</code> in the body	<code>Circle(this.radius);</code>
Read / change a field	<code>getRadius()</code> / <code>setRadius(20)</code>	<code>c.radius</code> / <code>c.radius = 20</code>
Private members	the <code>private</code> keyword (private to the class)	a leading <code>_</code> (private to the file)
Several constructors	overloading	named constructors, e.g. <code>Circle.create()</code>
Inheritance	<code>class B extends A</code>	<code>class B extends A</code>

This document is maintained by Dr. Mohamad AOUE
Last updated: 2026-09-21